December 5, 2025

# Predicting Personal Song Preference

*Final Project Summary*

*Group 15 - Sai Thar (23) and Tim Rioux (16)*

## Summary

This project uses existing data of a single user and a machine learning pipeline to predict a user's preference of "Like" or "Dislike" for new songs. The successful implementation of this pipeline provides a powerful tool in building a personalized recommendation engine, automatically suggesting the likelihood of the user's favorable songs, creating tailored playlists according to user's music style upon newly released songs.

This project uses two main datasets, daily tracks and library tracks. The daily tracks data contains a selection of new music that is regularly updated. The library tracks data is manually added to the collection by the user. The target variable, 'Like/Dislike', has three classes: "Like", "Dislike" and "Neutral". The goal of this project is to predict the binary classification whether new/unseen songs will be "Liked" or "Disliked" by the user.

## Problem Statement & Objectives

A user's music taste is complex and dynamic. The original datasets do not have a clear single indicator for "Like/Dislike" of a song. Therefore, we manually created a new "Like/Dislike" column with a calculation based on skip counts and play counts, which are included in the dataset. The thought process behind the new column is that if a user likes the song, the skip count is low and listens repetitively. We can classify a song as 'Like' if skip count divided by play count (skip ratio) is less than a threshold, which may be adjusted. The provided datasets capture rich listening behavior (End Reason Type, Date Played, Hours) and song metadata (Genre, Artist, Track Year) that can be used to learn a user's preferences.

The central problem is: **Can we build a model that accurately predicts whether a user will 'Like' or 'Dislike' a song, based on features *other than* the ones used to create the target?**

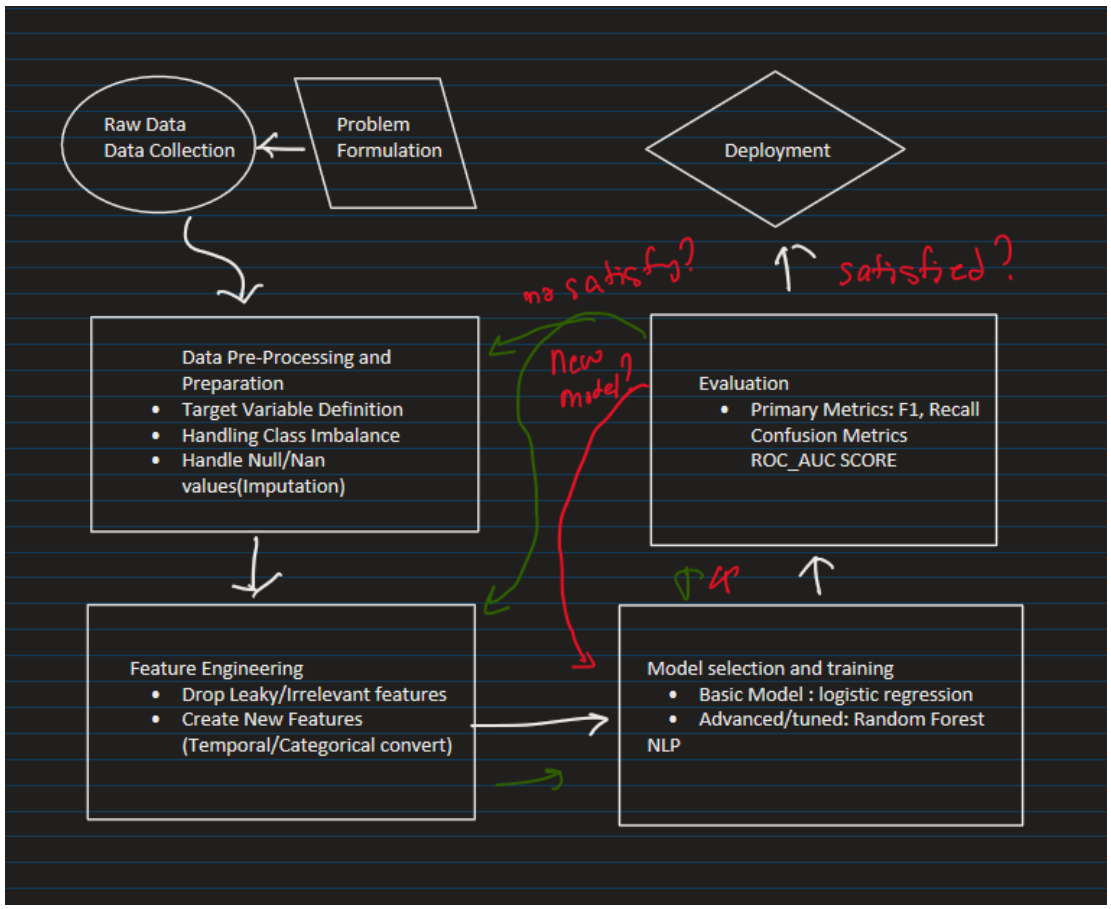
## Project Objectives

1. **Data Preprocessing:** Clean and prepare the dataset for modeling. This includes handling missing values, standardizing strings, and parsing temporal fields.

2. **Feature Engineering:** Systematically create and select features that have predictive power, encode categorical data, and scale all features as necessary. E**xplicitly avoid data leakage**.

3. **Model Development:** Train and tune a baseline (Logistic Regression) and an advanced (XGBoost) classification model.

4. **Model Evaluation:** Evaluate the models on a held-out test set using appropriate metrics (Accuracy, Precision, Recall, F1-Score, and ROC-AUC) to assess performance, paying close attention to the minority 'Dislike' class.



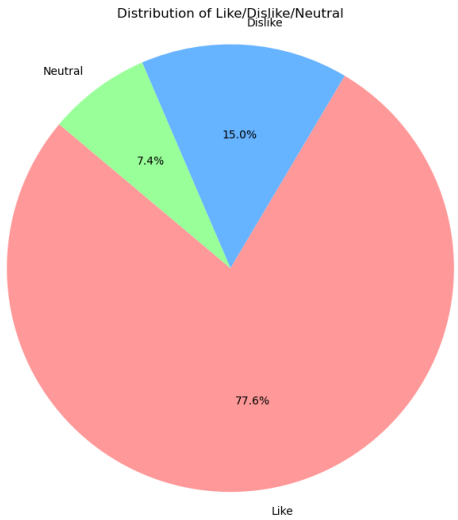
## Methodology

### 1. *Data Pre-processing and Preparation*

- **Base dataset:**
  - Joining the two main datasets, daily tracks and library tracks, results in roughly 58,000 rows/records and 68 features before pre-processing.
- **Target Variable Definition:**
  - The Like/Dislike column is the target, which is based on skip counts and play counts. The target is encoded as follows: Like = 1, Dislike = 0. The logic for calculating the Like/Dislike classification is shown below:

```
def cal_ratio(row):
    #assign values from each column
    play = row['Track Play Count']
    skip = row['Skip Count_y']

    #condition check --> only songs that play once will calculate
    if play >0:
        return skip/play
        # return the ratio if plays
    elif play ==0 and skip >0:
        # return total dislike if no play and skips
        return 1.0
    elif play ==0 and skip == 0:
        #no play, no skip, natural or never listen before
        return np.nan
    else:
        return np.nan

def Like_dislike (ratio):

    if pd.isna(ratio):
        return "Neutral"
    elif ratio < 0.5:
        return "Like"
    else:
        return "Dislike"

# function call to calculate ratio and set a column within df
df['Skip Ratio'] = df.apply(cal_ratio, axis=1)
# function call to set like/Neutral/dislike and set a column within df
df['Like/Dislike'] = df['Skip Ratio'].apply(Like_dislike)
```

- **Handling Class Imbalance:**
  - As can be seen from the pie chart below, the binary target is imbalanced.




- This is addressed by:
  - changing the threshold used the ratio of skip/play.
  - Using a stratified train/test split to ensure the same class distribution in both sets.
  - employing evaluation metrics (Precision, Recall, F1) that are robust to imbalance.

- **Handling Nan values**
  - There are many Nan values across the dataset, which are handled in the following manner:
    - drop if there's no meaningful contribution to the dataset.
    - replace with 0, mean or median if the feature column is important.

## 2. *Feature Engineering*

This is the most critical stage. Predictive features were separated from "leaky" features.

- **Leaky Features (To Be Dropped):**
  - Track Play Count and Skip Count_y: These columns were used to *create* the Like/Dislike target. Using them as features would cause severe data leakage, as the model would simply learn the Skip Ratio < Threshold rule and achieve a misleadingly perfect score. They are excluded from the feature set accordingly.

- **Irrelevant/Redundant Features (To Be Dropped):**
    - Media type: Mostly NaN values and all values are “Song” type except for four records being “video” type.
    - Title, Sort Name, Album Artist: These are high-cardinality text fields. While advanced NLP could be used, they are dropped in favor of Artist.
- **Features Engineered:**
    - **Temporal Features:**
        - The “Date Played” feature is converted from int64 (e.g., 20150704) to datetime objects to extract day_of_week, month, and year.
        - The “Hours” feature is cleaned as it's an object type with some values being integers and other values being a string of integers separated by commas. Therefore, this feature needs multiple steps to convert to numeric values that can be interpreted by an ML model. After initial cleaning, the values are binned into time_of_day ('Morning', 'Afternoon', 'Night') and encoded. Multi-hot encoding is used because records with multiple hours can be in multiple categories. The logic for the engineering of this feature is below:

### Engineering "Hours"

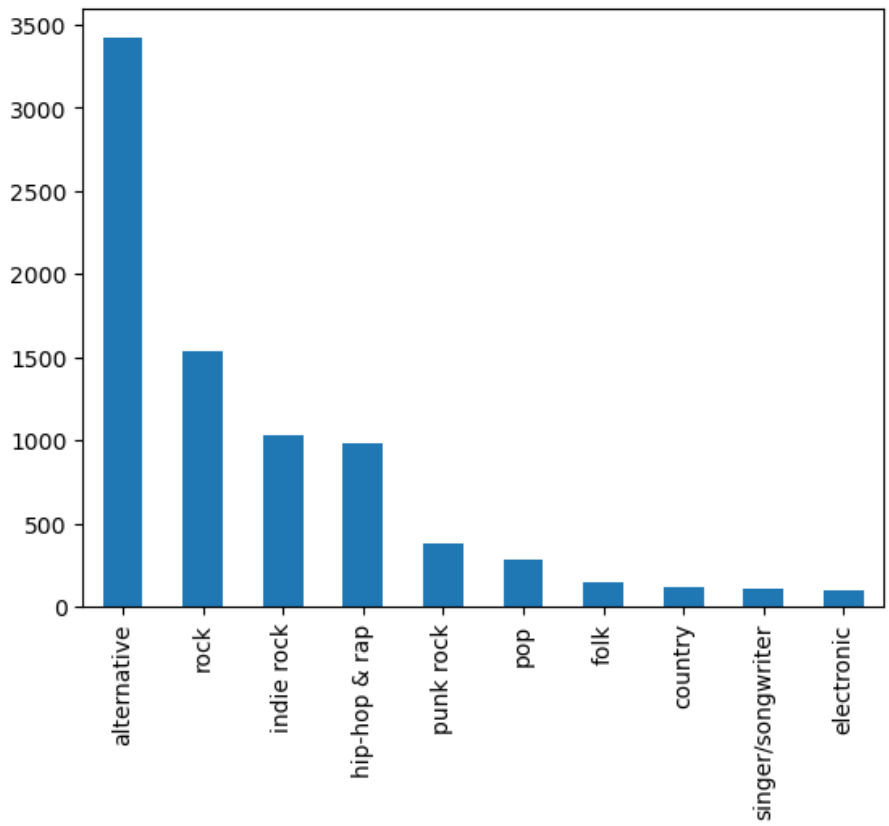
1. convert string into a list
2. Use function to categories integers into Morning, Afternoon, and Night categories
3. Multi-hot encode (because multiple categories can be 1.

```python
def encode_periods(hours):
    periods = {
        'Morning': 0,
        'Afternoon': 0,
        'Night': 0
    }

    for h in hours:
        if 5 <= h <= 11:
            periods['Morning'] = 1
        elif 12 <= h <= 16:
            periods['Afternoon'] = 1
        else:  # night = 17-23 OR 0-4
            periods['Night'] = 1

    return pd.Series(periods)

# convert Hours column to a list of integers
df['Hours_list'] = df['Hours'].apply(
    lambda x: list(map(int, str(x).split(',')))
)

# Apply multi-hot encoding
df[['Morning', 'Afternoon', 'Night']] = df['Hours_list'].apply(encode_periods)
df.loc[[1,29],['Morning', 'Afternoon', 'Night']].head(50)
```

| | Morning | Afternoon | Night |
|---|---|---|---|
| **1** | 0 | 0 | 1 |
| **29** | 0 | 1 | 1 |

- **Categorical Features:**
  - End Reason Type: Highly predictive (e.g., NATURAL_END_OF_TRACK vs. MANUALLY_SELECTED_PLAYBACK_OF_A_DIFF_ITEM). is one-hot encoded.
  - Artist, Album, Genre: These are key predictors and are encoded. Due to high cardinality, we used one-hot encoding for the top 25 most frequent categories and grouped the rest into an 'Other' category. A histogram (below) shows the distribution of the top 10 genres.

```
Number of unique Genres before Cleaning: 116
Number of unique Genres after Cleaning: 25
```




- **Numerical Features:**
  - Play Duration Milliseconds and other features of varying ranges were scaled using StandardScaler.
  - Track Year is treated as a numerical feature. Missing values are imputed with the median or mean year.

### 3. *Model Selection & Training*

A ColumnTransformer and Pipeline object will be used to ensure all transformations are applied consistently and to prevent data leakage from the test set. A logistic regression was attempted, but abandoned in favor of XGBoost:

- **Data Split:** The dataset (56,789 records) will be split to 80/10/10 (Train, Test, Validate) using stratification.
- **Baseline Model: Logistic Regression**. This provides a simple, interpretable baseline to measure against.
- **Advanced Model: XGBoost**. This model is robust, handles non-linear relationships well, and is excellent for tabular data. It will be tuned for key hyperparameters (e.g., n_estimators, max_depth).

### 4. *Evaluation*

Success is measured by the model's performance on the 10% held-out test set and 10% validation set.

For the initial set-up was with the default parameter with n_estimators = 300.

We use X_test sample as an initial test set to predict Y_test(true_prediction).

- **Primary Metrics:**
  - **F1-Score (Weighted):** A balance between Precision and Recall, providing a single score that accounts for imbalance.
  - **Recall (for 'Dislike' class):** This is critical. How many of the *actual* 'Dislike' songs did we correctly identify? A high recall here is crucial for a "clean-up" tool.
  - **Confusion Matrix:** To visualize *how* the model is making errors (e.g., is it misclassifying 'Dislike' as 'Like'?).

## Results

### 1. *Exploratory model selection – logistic regression*

We attempted to use a logistic regression model since the target and prediction is a binary classification. However, the results were not ideal with a pseudo-R-squared value of 0.0585 (see more results in the below screenshot). A low pseudo $R^2$ value is logical for human interaction data due to its complexity and also sparse genre encodings (i.e., rare occurrences of certain genres. Additional feature engineering could

be done to improve results from a logistic regression. However, it's clear this may not be the best model for this application, so we will focus our remaining efforts on XGBoost.

```
model = sm.Logit(Y_train, X_train)
result = model.fit_regularized()
print(result.summary())
```

```
Optimization terminated successfully    (Exit mode 0)
            Current function value: 0.6023850340500378
            Iterations: 190
            Function evaluations: 190
            Gradient evaluations: 190
                           Logit Regression Results
==============================================================================
Dep. Variable:                Is_like   No. Observations:                 6828
Model:                          Logit   Df Residuals:                     6800
Method:                           MLE   Df Model:                           27
Date:                Mon, 01 Dec 2025   Pseudo R-squ.:                 0.05850
Time:                        16:38:59   Log-Likelihood:                -4113.1
converged:                       True   LL-Null:                       -4368.6
Covariance Type:            nonrobust   LLR p-value:                 7.911e-91
==============================================================================================
                             coef    std err          z      P>|z|      [0.025      0.975]
----------------------------------------------------------------------------------------------
Is_entire_song_played       0.9994      0.055     18.235      0.000       0.892       1.107
Morning                     0.1680      0.136      1.235      0.217      -0.099       0.435
Afternoon                   0.3645      0.080      4.566      0.000       0.208       0.521
Night                       0.0532      0.057      0.941      0.347      -0.058       0.164
```

## 2. *XGBoost – Base model*

XGBoost evaluation metrics for the base model are shown below.

```
print("\n--- Validation Scores (Base model) ---")
print(f"Accuracy by default:    {accuracy_score(Y_test, y_pred):.4f}")
print(f"F1 Score:               {f1_score(Y_test, y_pred):.4f}")
print(f"AUC:                    {roc_auc_score(Y_test, y_pred_proba_val):.4f}")
```

```
--- Validation Scores (Base model) ---
Accuracy by default:    0.8042
F1 Score:               0.8780
AUC:                    0.8084
```

## 3. *XGBoost - Hyper Parmeter Tuning*

After the initial model training and evaluation with default parameters, we begin to tune the model with the wide range of parameters defined below.

```python
param_dist = {
    'n_estimators': [100, 200, 300, 400, 500],
    'learning_rate': [0.01, 0.05, 0.1, 0.2, 0.3],
    'max_depth': [3, 4, 5, 6, 8, 10],
    'min_child_weight': [1, 3, 5, 7],
    'gamma': [0, 0.1, 0.2, 0.3, 0.4],
    'subsample': [0.6, 0.7, 0.8, 0.9, 1.0],
    'colsample_bytree': [0.6, 0.7, 0.8, 0.9, 1.0],
    'scale_pos_weight': [1, 2, 5, 10]
}

xgb = XGBClassifier(objective='binary:logistic', eval_metric='auc')


random_search = RandomizedSearchCV(
    estimator=xgb,
    param_distributions=param_dist,
    n_iter=50,
    scoring='f1', # Optimize for AUC (or use 'f1' or 'accuracy')
    cv=5,              # 5-fold Cross-Validation
    verbose=1,
    random_state=42,
    n_jobs=-1          # Use all available cores
)

print("Starting Hyperparameter Tuning...")
random_search.fit(X_train, Y_train)

# 5. Get the best results
print(f"\nBest Parameters: {random_search.best_params_}")
print(f"Best Training AUC Score: {random_search.best_score_:.4f}")

# 6. Evaluate on Validation Set with best model
best_model = random_search.best_estimator_

y_pred_val = best_model.predict(X_val)
y_pred_proba_val = best_model.predict_proba(X_val)[:, 1]

print("\n--- Validation Scores (Tuned model) ---")
print(f"Accuracy by default:    {accuracy_score(Y_val, y_pred_val):.4f}")
print(f"F1 Score:               {f1_score(Y_val, y_pred_val):.4f}")
print(f"AUC:                    {roc_auc_score(Y_val, y_pred_proba_val):.4f}")
```

We use randomSearch from RandomizedSearchCV with the parameters.

True Negatives (Correctly identified Dislikes): 61.0000

False Positives (Predicted Like, actually Dislike): 24.0000

False Negatives (Predicted Dislike, actually Like): 131.0000

True Positives (Correctly identified Likes): 638.0000

Sensitivity or TPR: 82.9649%

There is a 82.9649% chance of correctly predicting if user likes song

Specificity or TNR: 71.7647%

There is a 71.7647% chance of predicting if user dislikes song

Precision: 96.3746%

A user likes a song, and the probablity that is correct is 96.3746%

Negative Predictive Value: 31.7708%

User dislikes a song, but there is a 31.7708% chance that is incorrect

False Positive Rate: 28.2353%

There is a 28.2353% chance that predicting a user likes a song is incorrect

False Negative Rate: 17.0351%

There is a 17.0351% chance a prediciton of a user diliking a song is incorrect.

False Discovery Rate: 3.6254%

Predicts a user likes a song, but there is a 3.6254% chance this is incorrect.

Accuracy: 81.8501%

Sensitivity or TPR: 82.9649%

Specificity or TNR: 71.7647%

Precision: 96.3746%

Negative Predictive Value: 31.7708%

False Positive Rate: 28.2353%

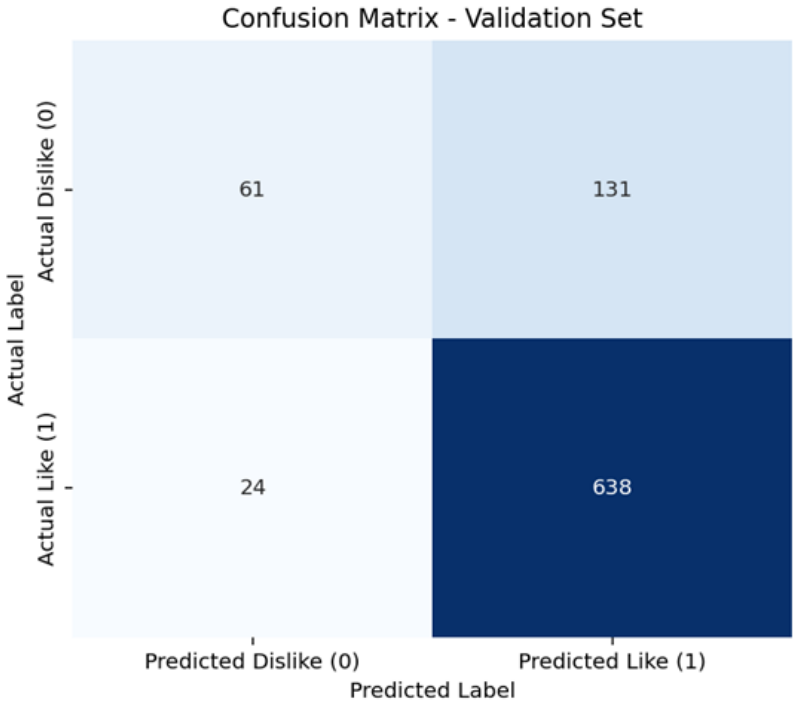
False Negative Rate: 17.0351%

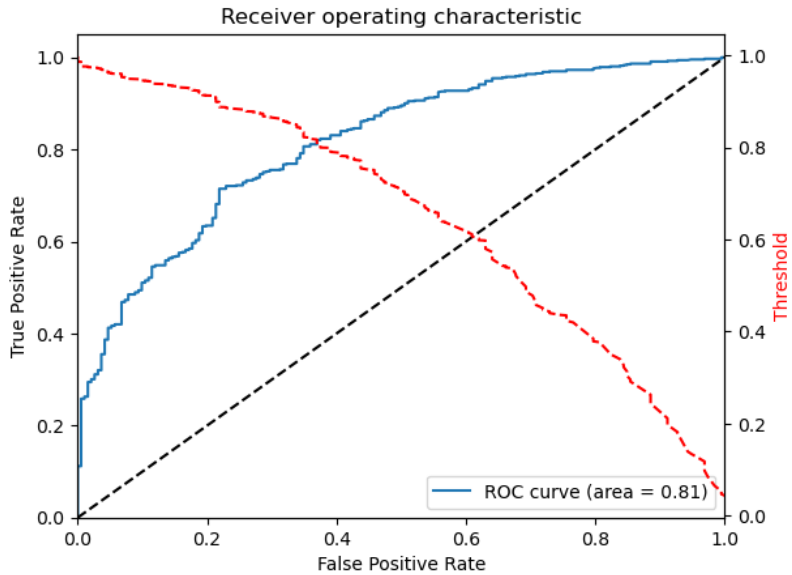
False Discovery Rate: 3.6254%

Accuracy: 81.8501%

Validation AUC 0.8044

```
--- Validation Scores (Tuned model) ---
Accuracy by default:    0.8150
F1 Score:               0.8895
AUC:                    0.8135
```






## Conclusion

In this project, we spent significant time evaluating large amounts of user listening data, cleaning and engineering the features and targets to have the best data for training and testing. We first attempted to use a logistic regression model based on the target type of a binary classification. Ultimately, this was not the correct model to use for this application, and we subsequently chose the more advanced XGBoost model to complete the project. The base model and tuned model provided similar evaluation metrics, which is perhaps an indication that the limitation is the data, given the complex nature of human interaction data. The AUC-ROC curve shows that the model provides good predictions (certainly better than random selection) but still has room for improvement. Two of the most important indicators of performance are Precision and False Positive Rate (FPR). A high precision, which we achieve 96% in the tuned model, means there is a 96% chance the user will like whatever song is provided to them. A low FPR is also desirable, which our tuned model had a relatively low value of 30%. In general, the lower the FPR, the less likely the user will skip a song. We believe this project provides a solid basis for predicting songs a user will like based on previous listening history.